

SISTEMAS OPERATIVOS

SEMANA 08 – SESIÓN 02

Unidad de aprendizaje 2:
Gestión de memoria y gestión de archivos



Universidad
Tecnológica
del Perú

Logro de aprendizaje

Al finalizar la sesión el estudiante logra:

- Administrar la memoria y los archivos que componen un sistema operativo.



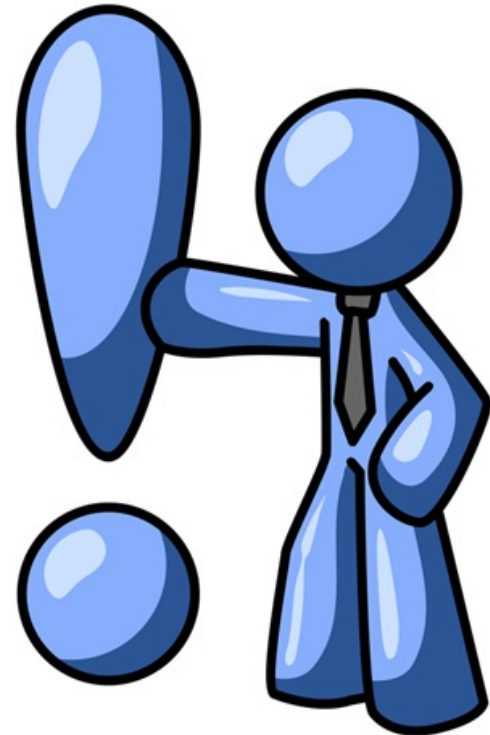
TEMARIO

- Gestión de Memoria.
- Administración básica de memoria.
- Asignación de memoria contigua.



Conocimientos previos

- Algoritmos



UTILIDAD

- Ejecución de programas cuyo requerimientos de memoria exceden a la memoria física.





ALGORITMOS DE REEMPLAZO

¿Por qué Necesitamos Algoritmos de Reemplazo?

Cuando la memoria física se llena y llega una nueva página, ¿cuál página existente debemos reemplazar para minimizar los fallos de página futuros?



ALGORITMOS DE REEMPLAZO

Objetivos de los Algoritmos

- **Minimizar fallos de página:** Reducir accesos a disco
- **Maximizar el rendimiento:** Mejorar tiempo de respuesta
- **Eficiencia computacional:** Bajo overhead del algoritmo
- **Justicia:** No penalizar procesos específicos



ALGORITMOS DE REEMPLAZO

Ejemplo de Fallo de Página:

Si un proceso solicita la página 7, pero solo hay marcos para 4 páginas y todos están ocupados, el SO debe decidir cuál página reemplazar.

 El algoritmo óptimo minimizaría los fallos de página, pero requiere conocimiento del futuro



Métricas de Evaluación

$$\text{Tasa de Fallos} = (\text{Número de Fallos de Página} / \text{Total de Referencias}) \times 100\%$$

$$\text{Tiempo Efectivo de Acceso} = (1 - p) \times \text{Tiempo_RAM} + p \times \text{Tiempo_Disco}$$



ALGORITMOS FIFO Y LRU

Algoritmo FIFO (First In, First Out)

Principio: La página que ha estado más tiempo en memoria es la primera en ser reemplazada.

Implementación: Cola simple que mantiene el orden de llegada.

Ventajas: Muy simple de implementar y entender.

Desventajas: Puede sufrir de la Anomalía de Belady.

Ejemplo FIFO: Secuencia: 7, 0, 1, 2, 0, 3, 0, 4 (3 marcos)

Paso 1:

7	-	-
---	---	---

 Fallo

Paso 2:

7	0	-
---	---	---

 Fallo

Paso 3:

7	0	1
---	---	---

 Fallo



Universidad
Tecnológica
del Perú



ALGORITMOS FIFO Y LRU

LRU (Least Recently Used)

Principio: Reemplaza la página que no ha sido usada por más tiempo.

Implementación: Pila o contador de tiempo para cada página.

Ventajas: Buen rendimiento, aproxima el comportamiento óptimo.

Desventajas: Mayor overhead computacional.



ALGORITMOS CLOCK Y ÓPTIMO

Algoritmo Clock (Segunda Oportunidad)

Principio: Aproximación a LRU usando un bit de referencia.

Funcionamiento:

1. Mantiene páginas en lista circular
2. Cada página tiene un bit de referencia
3. Si bit = 1: cambiar a 0 y continuar
4. Si bit = 0: seleccionar para reemplazo

Ventajas: Eficiente y buen rendimiento.

Página A
bit = 1



Página B
bit = 0



Página C
bit = 1



Puntero Clock



ALGORITMOS CLOCK Y ÓPTIMO

Algoritmo Óptimo (OPT/MIN)

Principio: Reemplaza la página que será referenciada más tarde en el futuro.

Características:

Teóricamente óptimo (mínimos fallos de página)

Imposible de implementar en la práctica

Usado como referencia para comparar otros algoritmos

Utilidad: Benchmark para evaluar algoritmos prácticos.



ALGORITMOS CLOCK Y ÓPTIMO

Algoritmo Óptimo (OPT/MIN)

Algoritmo	Complejidad	Rendimiento	Implementación
FIFO	$O(1)$	Regular	Muy simple
LRU	$O(1) - O(n)$	Bueno	Compleja
Clock	$O(1)$	Bueno	Simple
Óptimo	-	Perfecto	Imposible



PLANIFICACIÓN DE MEMORIA

¿Qué es la Planificación de Memoria?

Conjunto de algoritmos que determinan cómo asignar la memoria disponible a los procesos que la solicitan, optimizando el uso y minimizando la fragmentación.



PLANIFICACIÓN DE MEMORIA



Objetivos Principales

- **Minimizar fragmentación:** Reducir espacio desperdiciado
- **Maximizar utilización:** Usar eficientemente la memoria
- **Tiempo de respuesta:** Asignación rápida de memoria
- **Justicia:** Acceso equitativo a la memoria



Tipos de Fragmentación

Fragmentación
Externa

Espacios libres muy pequeños
entre procesos asignados

Fragmentación
Interna

Espacio desperdiciado
dentro de bloques asignados



Universidad
Tecnológica
del Perú



ALGORITMOS DE ASIGNACIÓN

Algoritmos Clásicos de Asignación

1 First Fit (Primer Ajuste)

Principio: Asigna el primer bloque libre que sea suficientemente grande.

Algoritmo:

```
for each free_block in memory_list: if free_block.size >= requested_size: allocate(free_block, requested_size) return success return failure
```

Ventajas: Rápido, simple de implementar

Desventajas: Puede crear fragmentación al inicio



ALGORITMOS DE ASIGNACIÓN

Algoritmos Clásicos de Asignación

2 Best Fit (Mejor Ajuste)

Principio: Encuentra el bloque libre más pequeño que satisfaga la solicitud.

Algoritmo:

```
best_block = null smallest_size = infinity
for each free_block in memory_list:
    if (free_block.size >= requested_size AND free_block.size < smallest_size):
        best_block = free_block
        smallest_size = free_block.size
if best_block != null:
    allocate(best_block, requested_size)
```

Ventajas: Minimiza el desperdicio de memoria

Desventajas: Más lento, puede crear muchos fragmentos pequeños



ALGORITMOS DE ASIGNACIÓN

Algoritmos Clásicos de Asignación

3 Worst Fit (Peor Ajuste)

Principio: Asigna el bloque libre más grande disponible.

Ventajas: Deja fragmentos grandes que pueden ser útiles

Desventajas: Desperdicia memoria, performance similar a Best Fit

Conclusiones

La abstracción básica para la gestión de memoria es la “dirección de memoria”.

El direccionamiento de memoria permite utilizar el concepto de memoria virtual, mecanismo que permite ejecutar programas que cuyos requerimientos de memoria exceden a la memoria física.

La paginación es un método que hace uso del concepto de memoria virtual. Los procesos utilizan “páginas” de memoria virtual, las cuales se corresponden con sus respectivos “marcos de página” de memoria física.

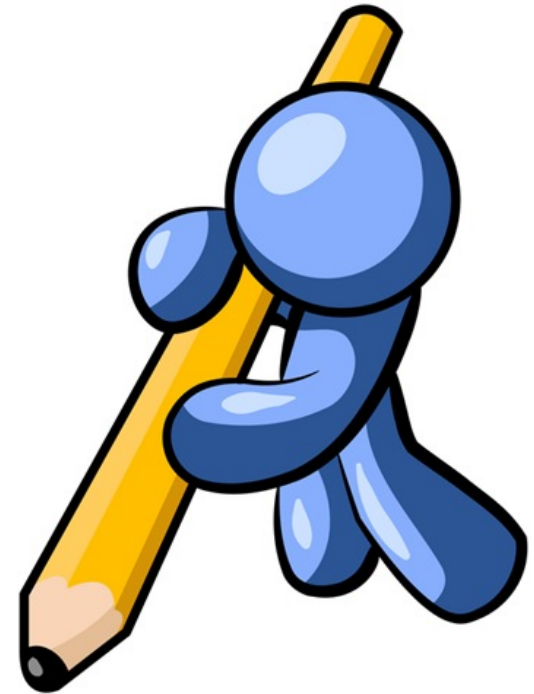
Preguntas

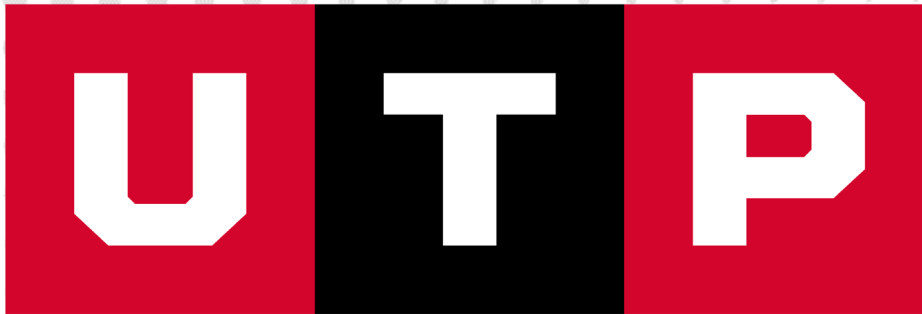
- ¿Tienes alguna duda o comentario?



CIERRE

- ¿Qué aprendiste en esta sesión?
- Te invitamos a compartir tus conclusiones en clase





**Universidad
Tecnológica
del Perú**